VINUNIVERSITY

Lab 01: Getting started with Java

Week 01, Feb 26 2026

College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring

Lab Practice Submission Instructions:

- This is an individual lab practice and will typically be assigned in the computer laboratory.
- Your program should work correctly on all inputs. If there are any specifications about how the program should be written (or how the output should appear), those specifications should be followed.
- Your code and functions/modules should be appropriately commented. However, try to avoid making your code overly busy (e.g., include a comment on every line).
- Variables and functions should have meaningful names, and code should be organized into functions/methods where appropriate.
- Academic honesty is required in all work you submit to be graded. You should **NOT** copy or share your code with other students to avoid plagiarism issues.
- Use the template provided to prepare your solutions (modified to provide your student ID, your name, etc).
- You should upload your .java file(s) to Codeforce before the end of the laboratory session unless the instructor gave a specified deadline. No Canvas or Codeforce submission will get 0 point.
- Submit a separate .java file for each lab problem, with the exact filename specified in each lab question. For example: **Lab1_P1_V202401744.java**.
- Late submission of lab practice without an approved extension will incur the following penalties:

  1. No submission by the deadline will incur 0.25 point deduction for each problem (most of the problems are due at the end of the lab session).
  2. The instructor will deduct an additional 0.25 point per problem for each day past the deadline.
  3. The penalty will be deducted until the maximum possible score for the lab practice reaches zero (0%) unless otherwise specified by the instructor.

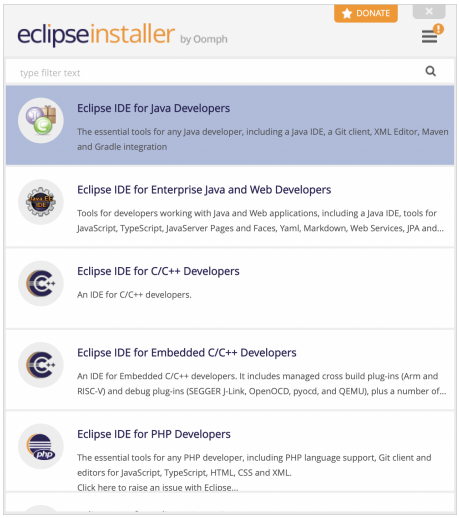

Figure 1: Get Eclipse IDE 2022-12.

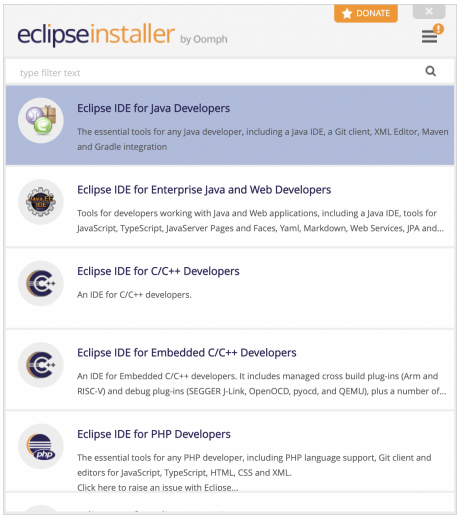

Figure 2: Select "Eclipse IDE for Java Developers".

Part 1: Setup Java Programming Environment

In this course, we are going to use "**Eclipse IDE 2022-12**" software as the main IDE in the practical labs and exams. Following we will follow the instructions to install the Eclipse IDE for Java Developers:

1 - There are many ways but the prefer way is to download from this link: `https://www.eclipse.org/downloads/` ; it should direct you to your corresponding OS system (Windows/MacOS/Linux).

2 - After download, run the file and select "**Eclipse IDE for Java Developers**" when asked (see Figure 2).

3 - Next you can select where Java installation is located. Note that you should have internet connection during the installation.

Note: You are welcome to use other IDEs as well (but you should also installed Eclipse IDE on your system just in case). Step by step to install Eclipse can be found the details below`https://www.eclipse.org/downloads/packages/installer`

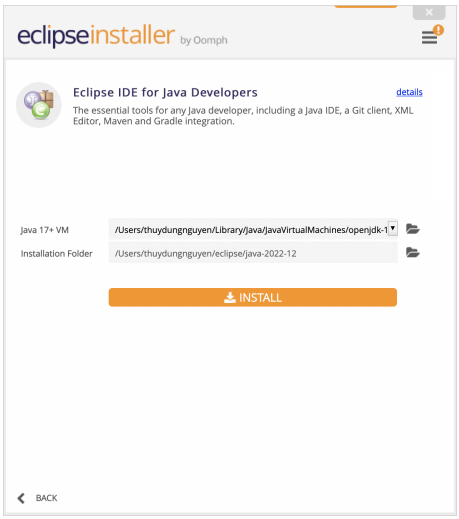

Figure 3: Choose the location and install the application.

Part 2: Create an Interactive Program

After installation, open the IDE (Eclipse/IntelliJ) and click **"Create a new Java project"**.

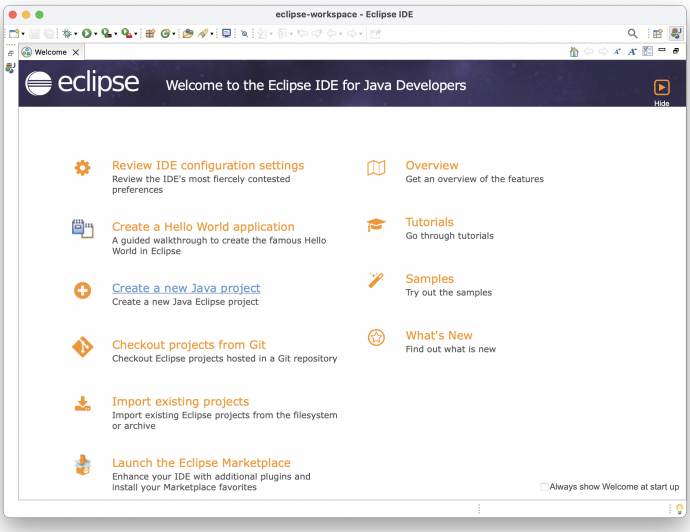

Figure 4: The IDE user interface.

Step 1: Project Setup

1. Create a new Java Project named `Lab1_Introduction`.
2. Right-click the `src` folder → **New** → **Class**.
3. Name the class **Greeting.java** and check the box for `public static void main(String[] args)`.

Step 2: Input and Display Task Instead of a static "Hello World", write a program that:

1. Prompts the user to enter their **Full Name**.
2. Prompts the user to enter their **Student ID**.
3. Prints a formatted welcome message.

Expected Console Output:

```
Enter your full name: Nguyen Van A
Enter your Student ID: 2024001
---------------------------------
Welcome to Java Lab 1!
Student: Nguyen Van A
ID: 2024001
---------------------------------
```

Part 3: Codeforces Submission Guide

For this course, we use Codeforces to automatically grade your lab assignments. Please follow these steps carefully to ensure your code is evaluated correctly:

1. **Login:** Go to `https://codeforces.com/` and log in with your account.
2. **Navigate to the Lab:** Open the specific link provided for this week's lab: `https://vinuni26oop.contest.codeforces.com/`.
3. **Enter the Contest:** Click on "Enter" for the current week's lab practice.
4. **Submit Code:**
   - Click on the **"Submit Code"** tab.
   - Select the correct **Problem** from the dropdown menu (e.g., "A - Greeting").
   - For **Language**, ensure you select **Java 11** (or the highest Java version available).
   - Click **"Choose File"** and upload your appropriately named `.java` file (e.g., `Lab1_P1_V202401744.java`).
   - Click **Submit**.
5. **Check Verdict:** Go to the "My submissions" tab to see if your code passed all test cases (**Accepted**) or if there are errors (e.g., **Wrong Answer**, **Compilation Error**).

Part 4: Reading Input (Python vs. Java)

Since you learned Python last semester, you are used to the simple `input()` function. Java requires a bit more setup using the `Scanner` class, which must be imported and instantiated as an object.

Python Equivalent:

```python
# Python reads everything as a string, requiring explicit casting
age = int(input("Enter an integer: "))
gpa = float(input("Enter a decimal number: "))
name = input("Enter your full name: ")
```

Java Implementation:

```java
import java.util.Scanner; // 1. Must import the class

public class InputExample {
    public static void main(String[] args) {
        // 2. Create the Scanner object
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter an integer: ");
        int age = sc.nextInt(); // Reads an integer directly

        System.out.print("Enter a decimal number: ");
        double gpa = sc.nextDouble(); // Reads a double directly

        // IMPORTANT NOTE: If you use nextLine() after nextInt() or nextDouble(),
        // you must add an extra sc.nextLine() to consume the leftover newline character.
        sc.nextLine();

        System.out.print("Enter your full name: ");
        String name = sc.nextLine(); // Reads a full line of text

        sc.close(); // 3. Always close the scanner when done
    }
}
```

Part 5: Conditional Statements

The logic of conditional statements remains the same, but Java uses parentheses `()` for conditions and curly braces `{}` to define code blocks instead of Python's indentation.

1. If-Else Statements

Python Equivalent:

```python
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
else:
    print("Grade: C or lower")
```

Java Implementation:

```java
int score = 85;

if (score >= 90) {
    System.out.println("Grade: A");
} else if (score >= 80) { // Java uses 'else if' instead of 'elif'
    System.out.println("Grade: B");
} else {
    System.out.println("Grade: C or lower");
}
```

2. Switch Statements

Note on Python: Older versions of Python (before 3.10) do not have a `switch` statement (you had to use long `if-elif` chains). Python 3.10 introduced `match-case`, which is similar. Java has had the `switch` statement from the beginning.

Java Implementation:

```java
int day = 3;

switch (day) {
    case 1:
        System.out.println("Monday");
        break; // Crucial: prevents falling through to the next case
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default: // Similar to 'else'
        System.out.println("Invalid day");
}
```

Part 6: Loops in Java

While Python's loops heavily rely on iterating over sequences (like lists or `range`), Java gives you granular control over the loop's starting point, condition, and increment step.

1. The for Loop

Python Equivalent:

```python
# Prints numbers from 1 to 5
for i in range(1, 6):
    print("Count:", i)
```

Java Implementation:

```java
// Structure: for (initialization; condition; update)
for (int i = 1; i <= 5; i++) {
    System.out.println("Count: " + i);
}
```

2. The while Loop

The `while` loop logic is almost identical in both languages, save for the syntax formatting.

Python Equivalent:

```python
count = 1
while count <= 3:
    print("While loop count:", count)
    count += 1
```

Java Implementation:

```java
int count = 1;
while (count <= 3) {
    System.out.println("While loop count: " + count);
    count++; // Equivalent to count += 1
}
```

3. The do-while Loop

Note on Python: Python does NOT have a native `do-while` loop. In Python, you have to fake it using a `while True:` loop and a `break` condition.

Java has a built-in `do-while` loop. It guarantees the code block will execute **at least once**, because the condition is checked at the *end* of the loop instead of the beginning.

Java Implementation:

```java
int num = 10;
do {
    // This prints at least once, even though 10 is not less than 5.
    System.out.println("This prints at least once. Num is: " + num);
    num++;
} while (num < 5);
```